

Лабораторна робота №3

Робота із базами даних в .NET. Використання бібліотеки Entity Framework.

Виконав студент групи 302-ТК

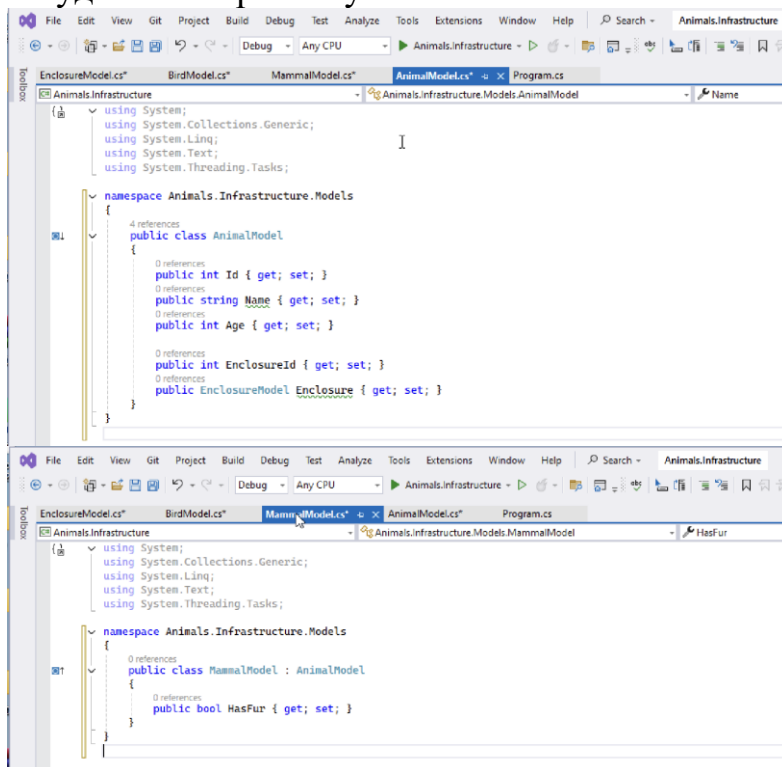
Рогочий Михайло

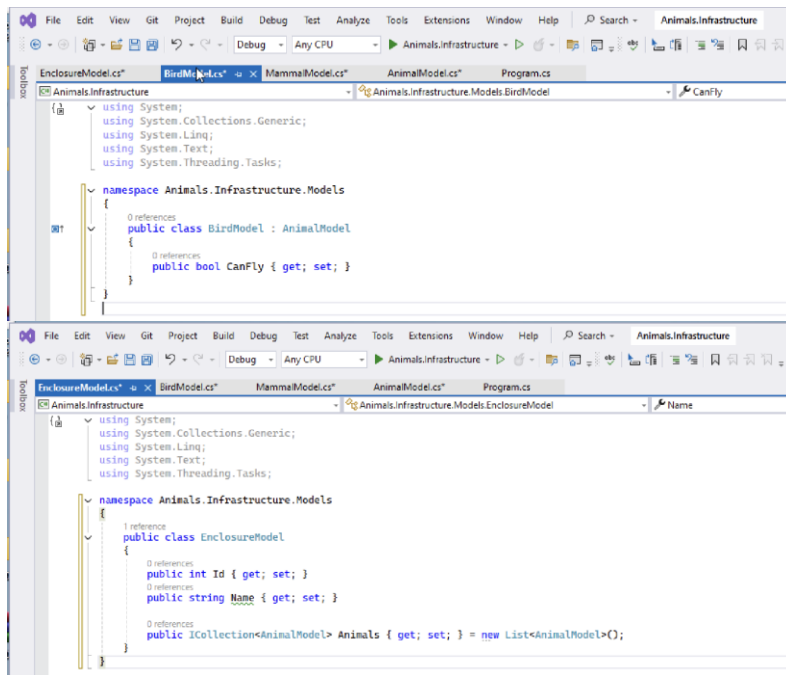
Передумова:

Завдання

1. Створити проєкт {Назва тематики}.Infrastructure. У новоствореному проєкті створити папку Models. У створеній папці додати модель (набір класів), яка відповідає створеній у першій лабораторній роботі моделі, та містить всередині себе властивості. Наприклад, якщо існував у першій лабораторній клас Bus, у папці Models необхідно створити клас BusModel. Отримана модель має обов'язково мати зв'язки один-до-одного та один-до-багатьох. За додавання зв'язків багато-до-багатьох можна отримати додаткові бали. Для наслідування більш пріоритетно застосовувати підхід Таблиця на тип (Table-per-Type).

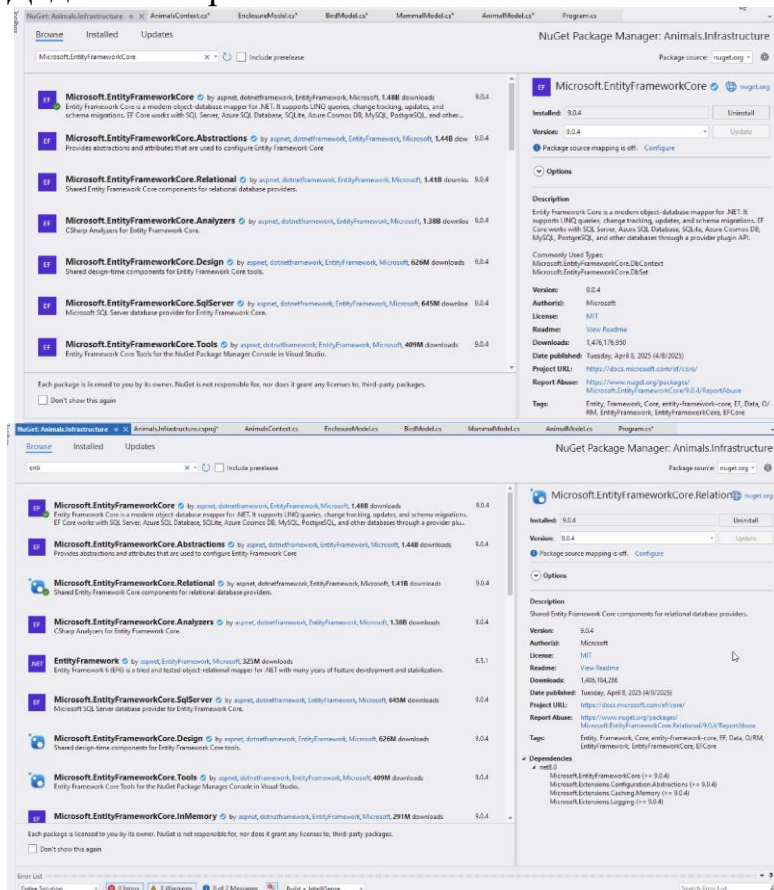
Оскільки у першій лабораторній роботі в мене була тематика Тварини, то її будемо використовувати.

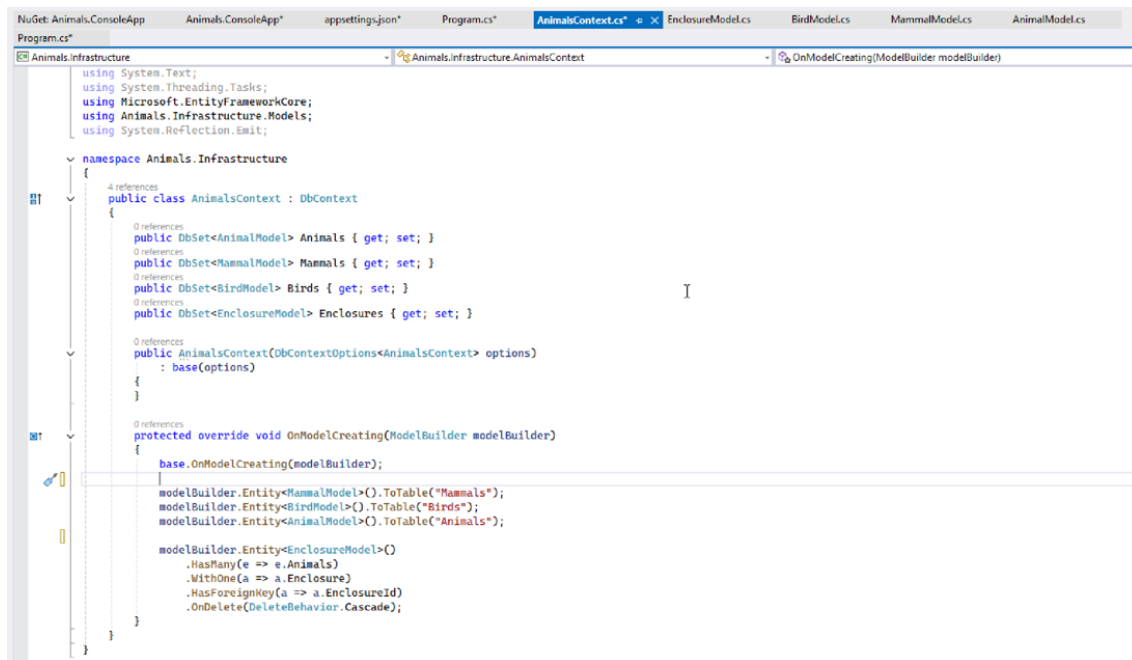




2. Створити клас *{Назва тематики}Context*, який наслідуватиметься від класу `DbContext` та опише схему бази даних для обраної тематики застосовуючи Entity Framework та анотації або Fluent API на вибір, Fluent API більш пріоритетно.

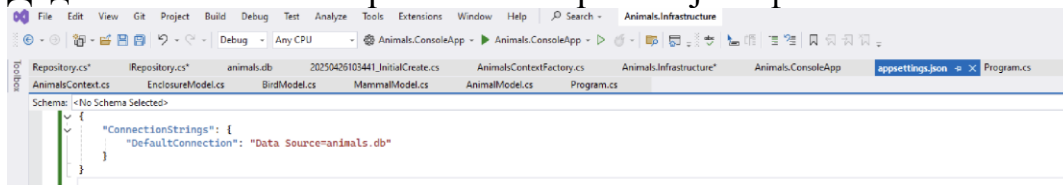
Додаємо потрібні бібліотеки



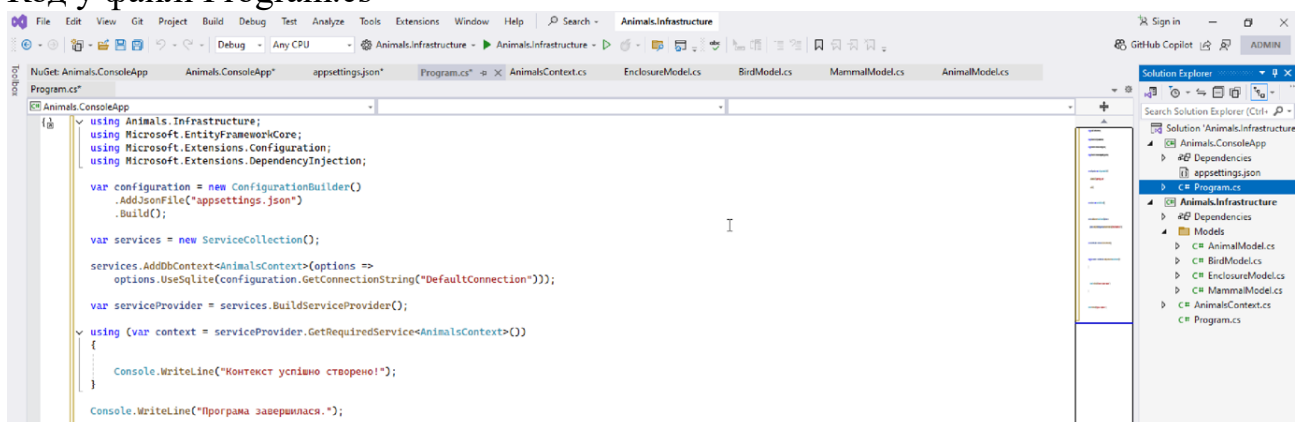


3. Створити міграцію використовуючи Entity Framework. Створену міграцію застосувати до реляційної бази даних. Можна використовувати будь-яку реляційну СУБД, як MS SQL, PostgreSQL, MySQL, тощо. Також слід використати SQLite, коли немає можливості розробляти та здавати Лабораторну роботу на одній системі. При використанні SQLite файл бази даних необхідно додати до гіт репозиторія.

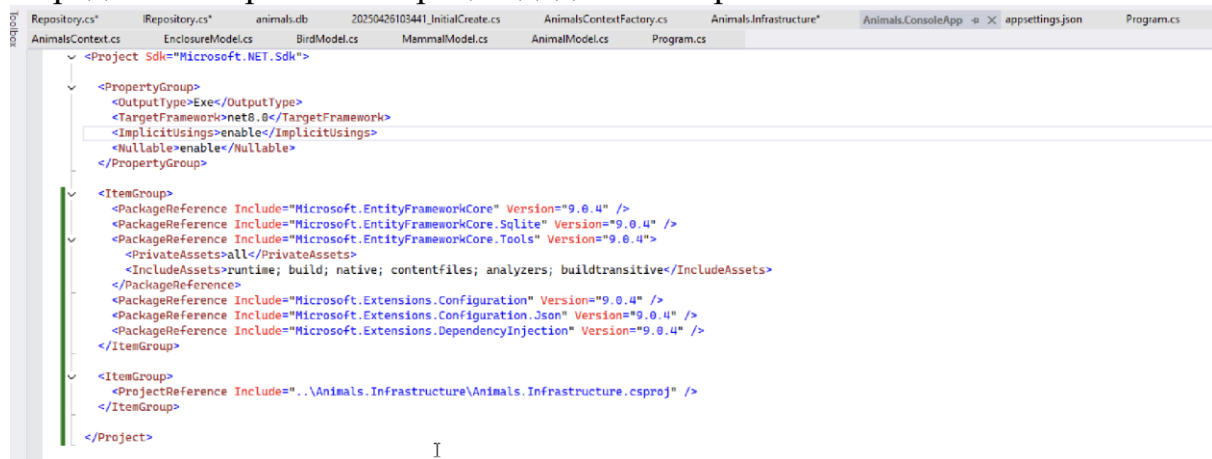
Додаємо консольний проект та створюємо json файл



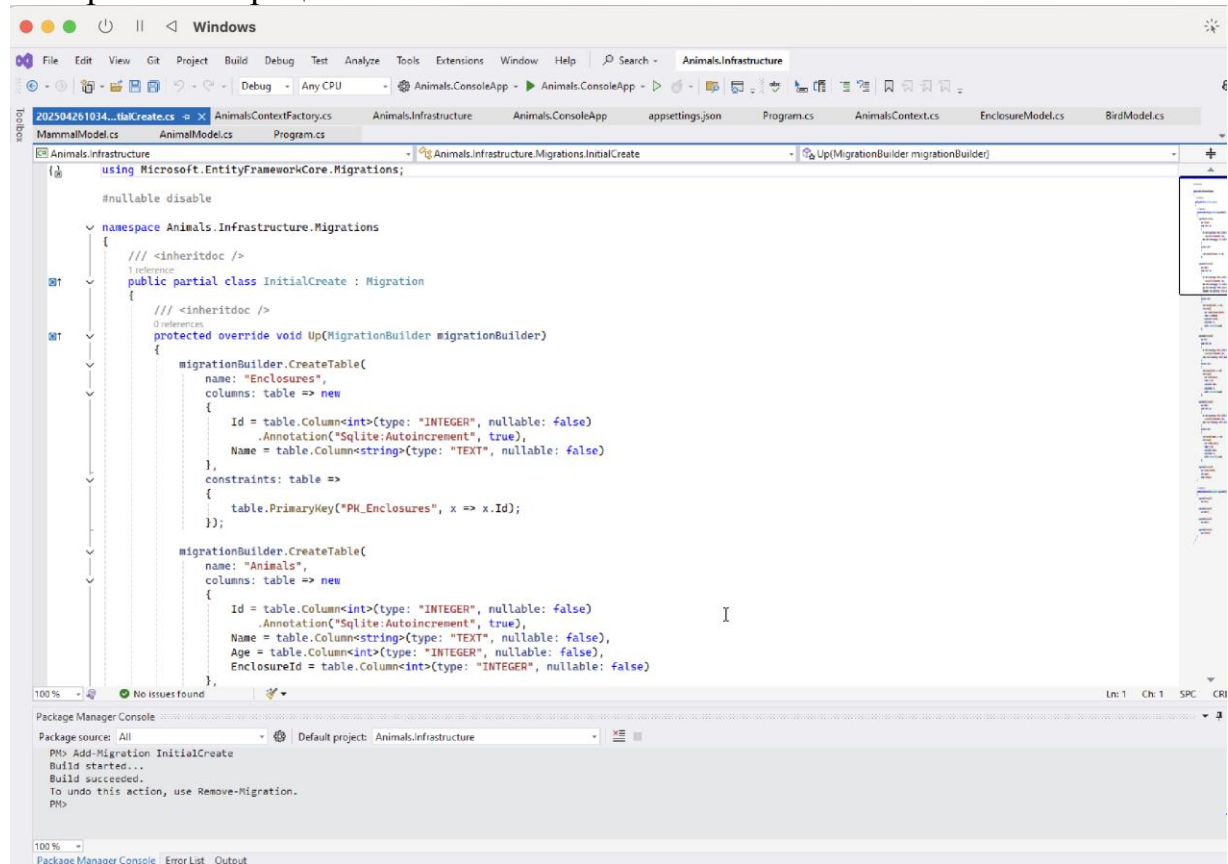
Код у файлі Program.cs



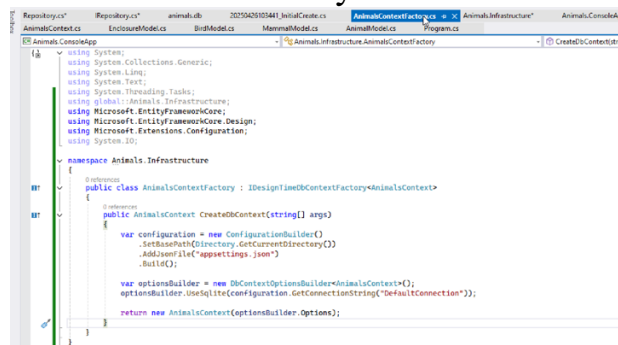
Перед тим як зробити міграцію додаємо потрібні пакети



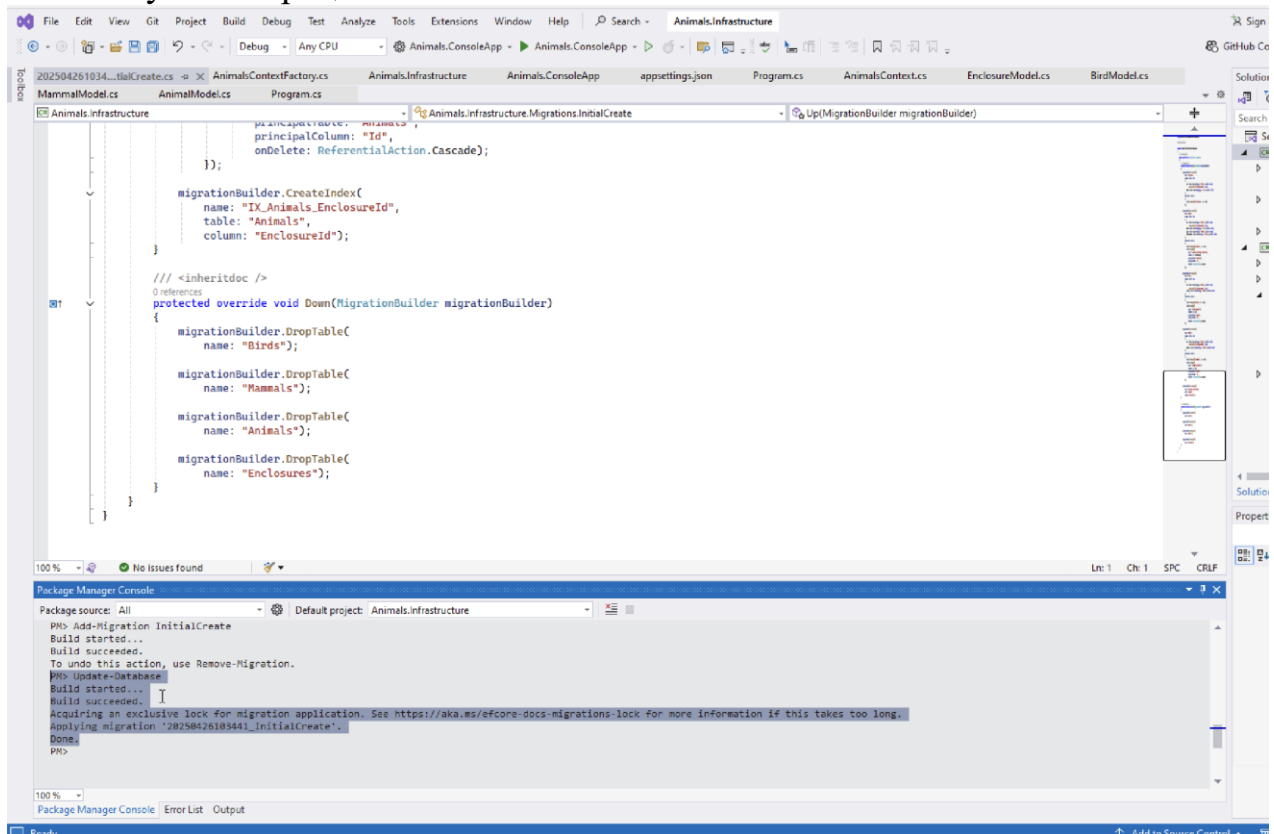
Створюємо міграцію



Для коректного виконання міграції також створюємо спеціальний клас AnimalContextFactory.cs

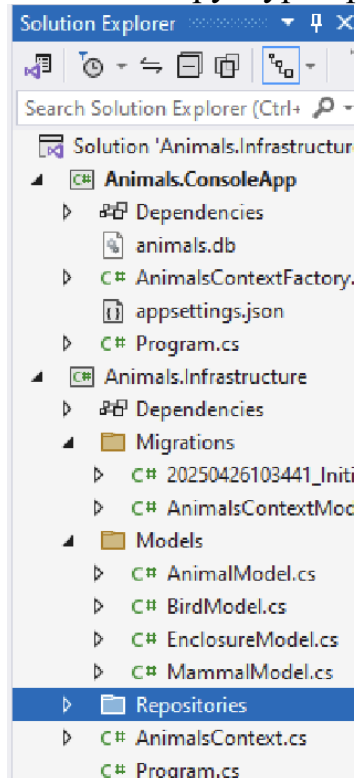


Застосовуємо міграцію



Створилася база даних animals.bd

Нинішня структура проекту

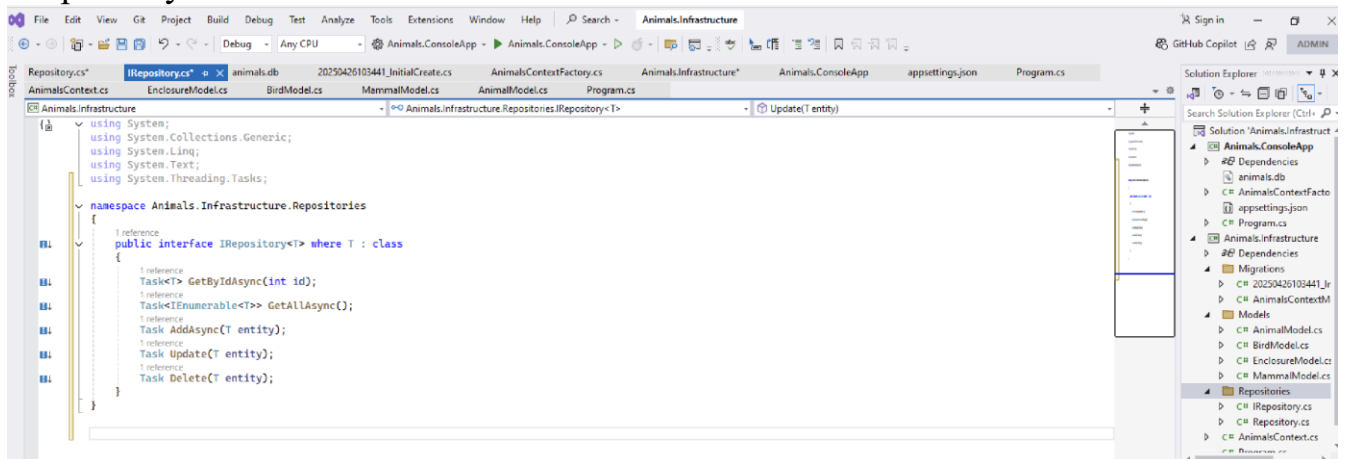


*Папка Repositories це вже наступний крок

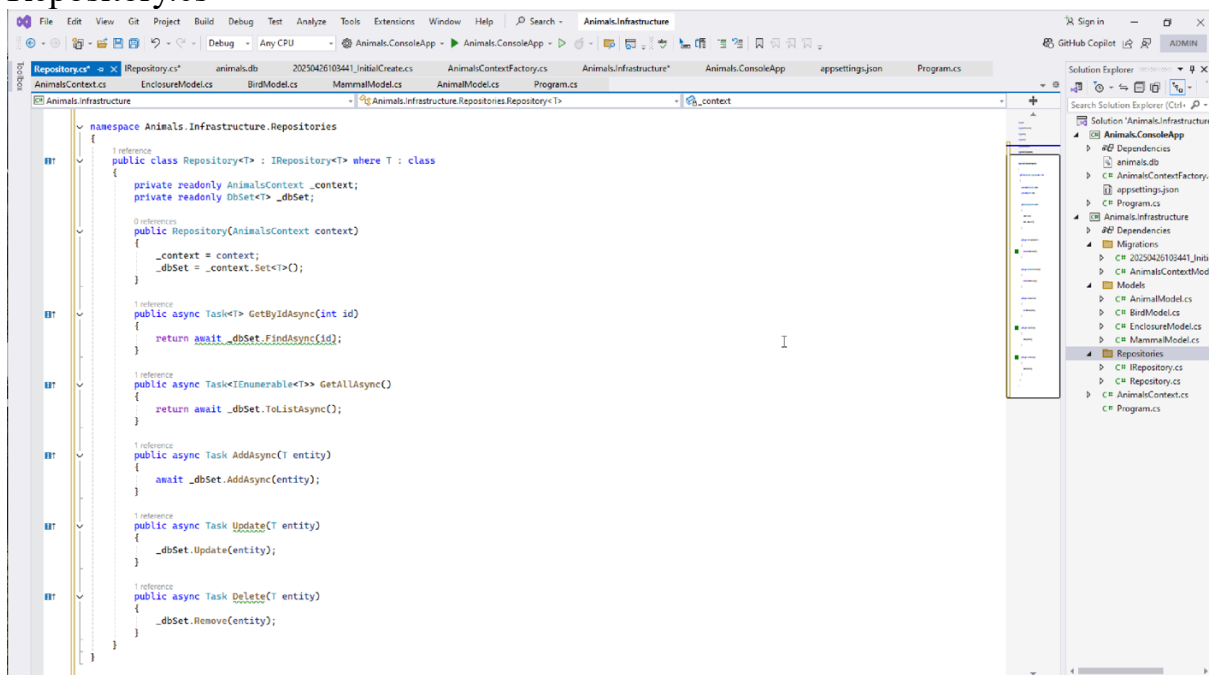
4.Реалізувати шаблон проєктування Репозиторій, *{Назва тематики}* Context який буде використовувати клас *{Назва тематики}* Context для досьугу до даних із БД:

```
public interface IRepository<T> where T : class
{
    Task<T> GetByIdAsync(int id);
    Task<IEnumerable<T>> GetAllAsync();
    Task AddAsync(T entity);
    Task Update(T entity);
    Task Delete(T entity);
}
```

Створюю папку для репозиторію Repositories та створюю у ній два класи: IRepository.cs



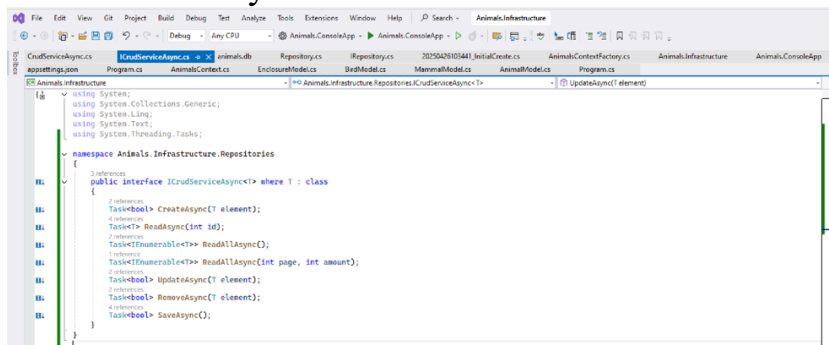
Repository.cs



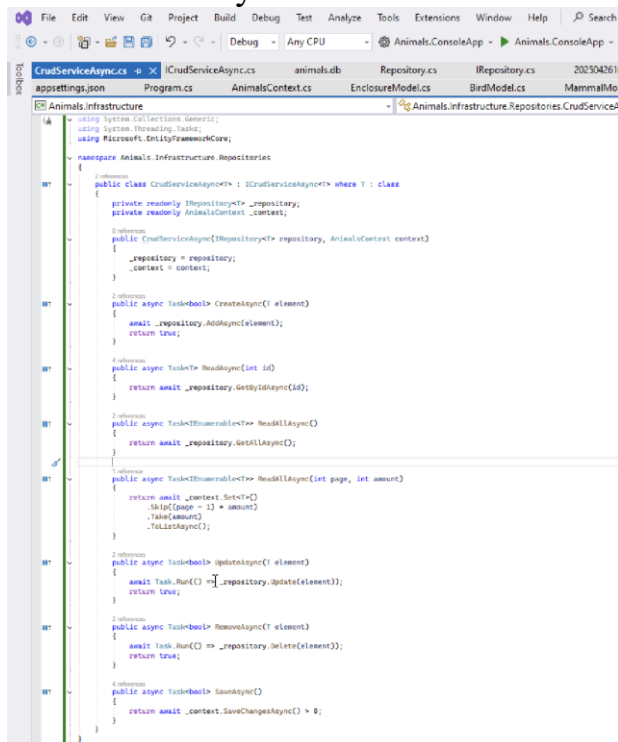
5.Оновити асинхрону версію дженерік CRUD сервісу, щоб він використовував репозиторій для доступу до даних:

```
public interface ICrudServiceAsync<T>
{
    public Task<bool> CreateAsync(T element);
    public Task<T> ReadAsync(Guid id);
    public Task<IEnumerable<T>> ReadAllAsync();
    public Task<IEnumerable<T>> ReadAllAsync(int page, int amount);
    public Task<bool> UpdateAsync(T element);
    public Task<bool> RemoveAsync(T element);
    public Task<bool> SaveAsync();
}
```

У вже існуючій папці Repositories створюємо два файли:
ICrudServiceAsync.cs

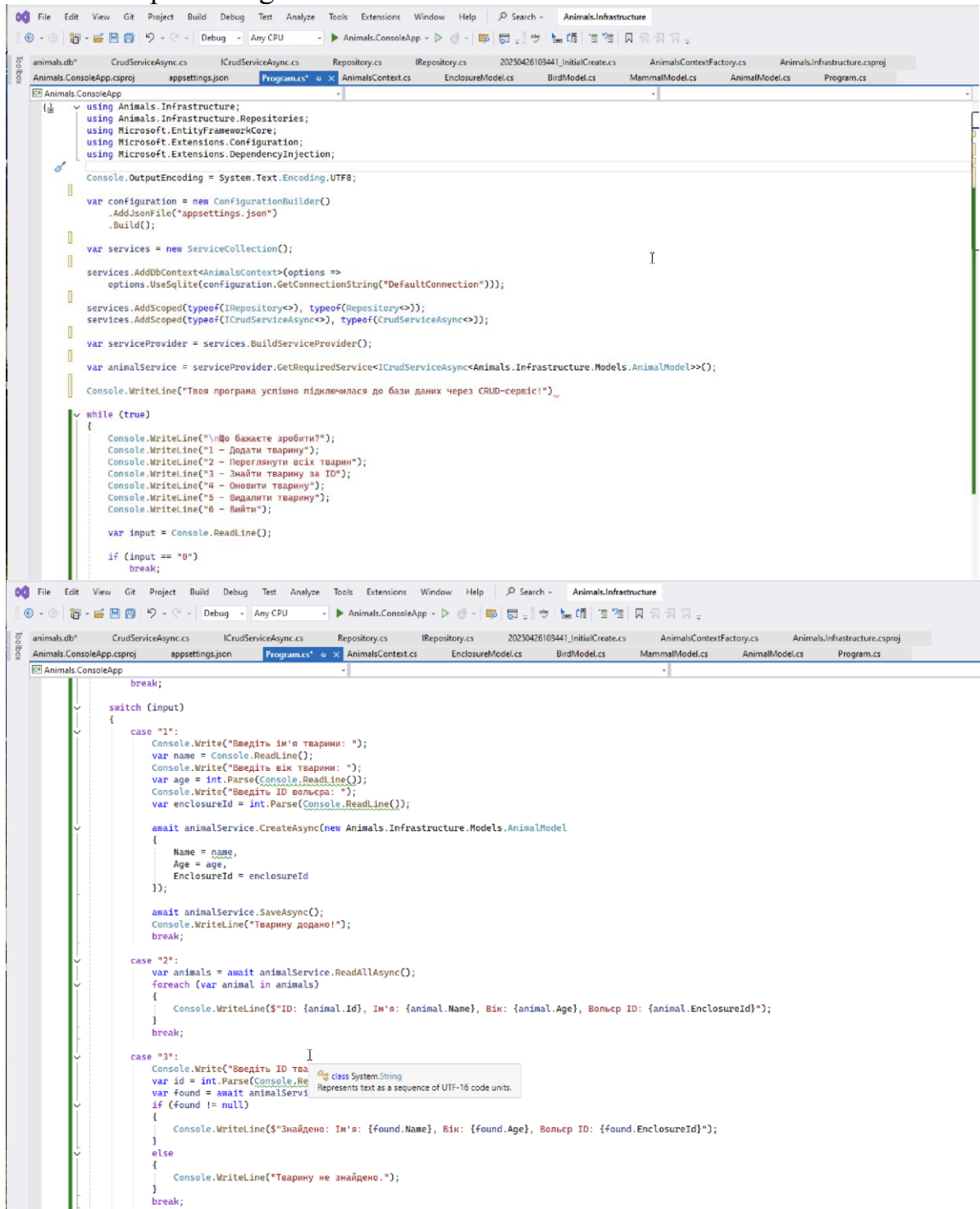


CrudServiceAsync.cs



6. Модифікуйте консольний застосунок, щоб він використовував оновлену версію CRUD сервісу та дані із бази даних.

Змінюємо файл Program.cs



```
using Animals.Infrastructure;
using Animals.Infrastructure.Repositories;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;

Console.OutputEncoding = System.Text.Encoding.UTF8;

var configuration = new ConfigurationBuilder()
    .AddJsonFile("appsettings.json")
    .Build();

var services = new ServiceCollection();

services.AddDbContext<AnimalsContext>(options =>
    options.UseSqlite(configuration.GetConnectionString("DefaultConnection")));

services.AddScoped(typeof(IRepository<>), typeof(Repository<>));
services.AddScoped(typeof(ICrudServiceAsync<>), typeof(CrudServiceAsync<>));

var serviceProvider = services.BuildServiceProvider();

var animalService = serviceProvider.GetRequiredService<ICrudServiceAsync<Animals.Infrastructure.Models.AnimalModel>>();

Console.WriteLine("Твоя програма успішно підключилася до бази даних через CRUD-сервіс!");

while (true)
{
    Console.WriteLine("\nЩо бажаєте зробити?");
    Console.WriteLine("1 - Додати тварину");
    Console.WriteLine("2 - Переглянути всіх тварин");
    Console.WriteLine("3 - Знайти тварину за ID");
    Console.WriteLine("4 - Оновити тварину");
    Console.WriteLine("5 - Видалити тварину");
    Console.WriteLine("6 - Вийти");

    var input = Console.ReadLine();

    if (input == "0")
        break;

    switch (input)
    {
        case "1":
            Console.WriteLine("Введіть ім'я тварини: ");
            var name = Console.ReadLine();
            Console.WriteLine("Введіть вік тварини: ");
            var age = int.Parse(Console.ReadLine());
            Console.WriteLine("Введіть ID вольєра: ");
            var enclosureId = int.Parse(Console.ReadLine());

            await animalService.CreateAsync(new Animals.Infrastructure.Models.AnimalModel
            {
                Name = name,
                Age = age,
                EnclosureId = enclosureId
            });

            await animalService.SaveAsync();
            Console.WriteLine("Тварину додано!");
            break;

        case "2":
            var animals = await animalService.ReadAllAsync();
            foreach (var animal in animals)
            {
                Console.WriteLine($"ID: {animal.Id}, Ім'я: {animal.Name}, Вік: {animal.Age}, Вольєр ID: {animal.EnclosureId}");
            }
            break;

        case "3":
            Console.WriteLine("Введіть ID тварини: ");
            var id = int.Parse(Console.ReadLine());
            var found = await animalService.FindAsync(id);
            if (found != null)
            {
                Console.WriteLine($"Знайдено: Ім'я: {found.Name}, Вік: {found.Age}, Вольєр ID: {found.EnclosureId}");
            }
            else
            {
                Console.WriteLine("Тварину не знайдено.");
            }
            break;
    }
}
```



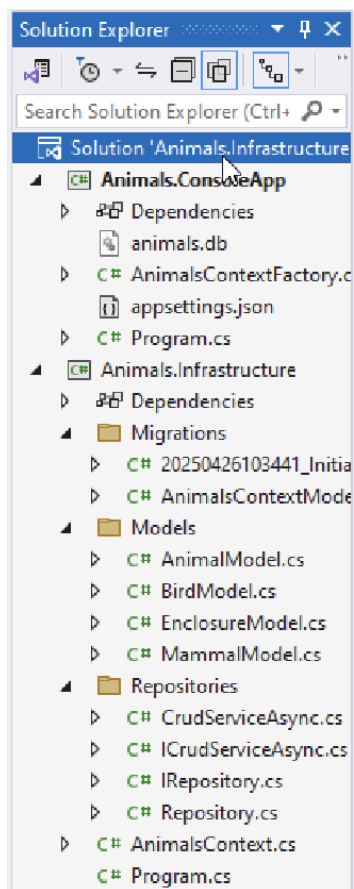
```
case "4":
    Console.WriteLine("Введіть ID тварини для оновлення: ");
    var updateId = int.Parse(Console.ReadLine());
    var animalToUpdate = await animalService.ReadAsync(updateId);
    if (animalToUpdate != null)
    {
        Console.WriteLine("Нове ім'я: ");
        animalToUpdate.Name = Console.ReadLine();
        Console.WriteLine("Новий вік: ");
        animalToUpdate.Age = int.Parse(Console.ReadLine());
        Console.WriteLine("Новий ID вольєра: ");
        animalToUpdate.EnclosureId = int.Parse(Console.ReadLine());

        await animalService.UpdateAsync(animalToUpdate);
        await animalService.SaveAsync();
        Console.WriteLine("Тварину оновлено!");
    }
    else
    {
        Console.WriteLine("Тварину не знайдено.");
    }
    break;

case "5":
    Console.WriteLine("Введіть ID тварини для видалення: ");
    var deleteId = int.Parse(Console.ReadLine());
    var animalToDelete = await animalService.ReadAsync(deleteId);
    if (animalToDelete != null)
    {
        await animalService.RemoveAsync(animalToDelete);
        await animalService.SaveAsync();
        Console.WriteLine("Тварину видалено!");
    }
    else
    {
        Console.WriteLine("Тварину не знайдено.");
    }
    break;

default:
    Console.WriteLine("Невідома команда.");
    break;
```

Кінцева структура проекту:



The screenshot displays the Visual Studio IDE with a C# console application open. The code implements a simple zoo management system with the following logic:

- Menu Options:**
 - 1 - Add animal (Додати тварину)
 - 2 - View all animals (Переглянути всіх тварин)
 - 3 - Find animal by ID (Знайти тварину за ID)
 - 4 - Delete animal (Видалити тварину)
 - 5 - Exit (Вийти)
- Animal Class:**
 - Attributes: Name (ім'я), Age (вік), EnclosureID (номер ID).
 - Methods: AddAnimal, ViewAllAnimals, FindAnimal, DeleteAnimal.
- Program Flow:**
 - Initializes an `AnimalContext` and a `Program` object.
 - Enters a loop where it prompts the user for a menu choice.
 - Based on the choice, it calls the corresponding `Animal` class method.
 - After each operation, it prompts the user to continue (Y/N).

The right sidebar shows the 'Diagnostic Tools' window, which provides a summary of events and resource usage during the application's execution.

Відкрив таблицю за допомогою додатку DB Browser for SQLite

